LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 5 – REKURSIF

NAMA : AIS NURHANIFAH

NIM : 109082500162

KELAS : S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Rekursi adalah sebuah paradigma pemecahan masalah dalam pemrograman di mana sebuah fungsi menyelesaikan tugasnya dengan memanggil dirinya sendiri pada input yang lebih kecil. Dalam bahasa Go, setiap kali fungsi rekursif dipanggil, sebuah *stack frame* baru dialokasikan di dalam memori untuk menyimpan variabel lokal dan parameter fungsi tersebut. Proses ini menciptakan rantai eksekusi yang terus menumpuk hingga program mencapai *base case*, yaitu suatu kondisi terminasi yang menghentikan pemanggilan berulang agar tidak terjadi *stack overflow* atau penggunaan memori yang tidak terkendali.

Struktur utama dari sebuah fungsi rekursif terdiri dari dua bagian yang saling melengkapi: *base case* dan *recursive step*. *Base case* merupakan fondasi yang memberikan jawaban langsung untuk kasus paling sederhana, seperti saat variabel mencapai angka 1 pada baris bilangan atau angka 0 pada operasi perpangkatan. Sementara itu, *recursive step* bertanggung jawab untuk mengarahkan program menuju *base case* tersebut dengan cara mengubah argumen fungsi pada setiap pemanggilan, misalnya dengan mengurangi nilai n sebesar 1 (n-1) atau menambah nilai pencacah (i+1) secara bertahap.

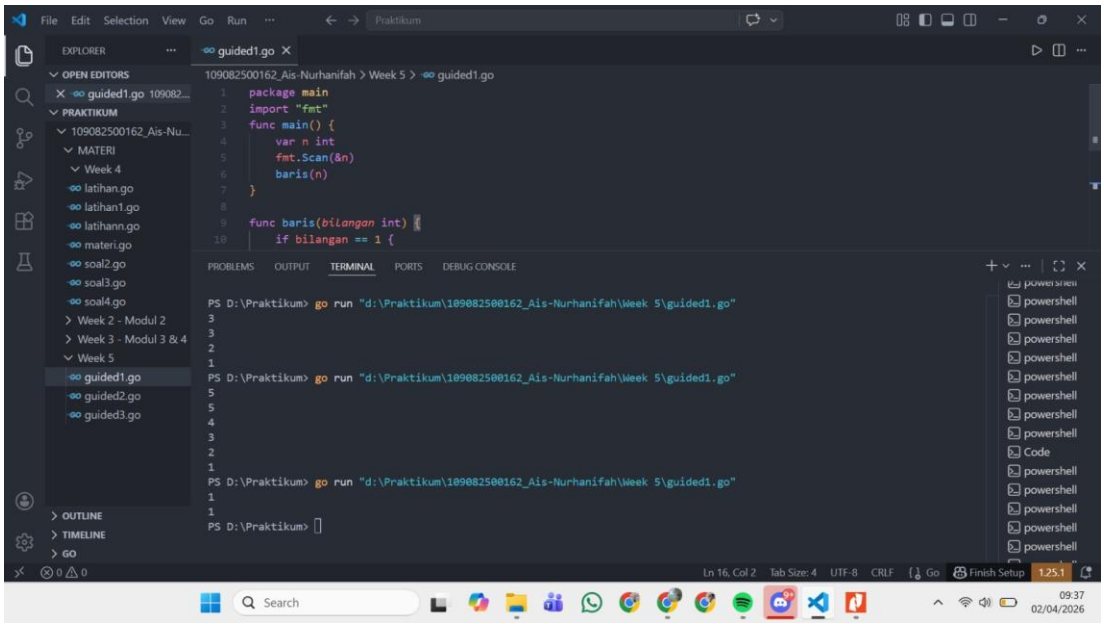
Selain kesederhanaan kodenya, rekursi sangat efektif untuk mengimplementasikan rumus matematis yang secara alami bersifat rekursif, seperti deret Fibonacci (Sn = S{n-1} + S{n-2}) atau faktorial. Berbeda dengan pendekatan iteratif yang menggunakan perulangan for atau while secara eksplisit, rekursi memanfaatkan tumpukan pemanggilan fungsi untuk menyimpan status perhitungan sementara. Meskipun demikian, pengembang harus memastikan bahwa setiap langkah rekursif membawa program selangkah lebih dekat ke kondisi berhenti agar seluruh proses dapat diselesaikan dengan akurat dan efisien.

B. Guided

1. [barisBilangan.go]

```
      package main
      import "fmt"
      func main() {
            var n int
            fmt.Scan(&n)
            baris(n)
      }
      func baris(bilangan int) {
            if bilangan == 1 {
                  fmt.Println(1)
            }else{
                  fmt.Println(bilangan)
                  baris(bilangan - 1)
            }
}
```

Output :



Penjelasan :

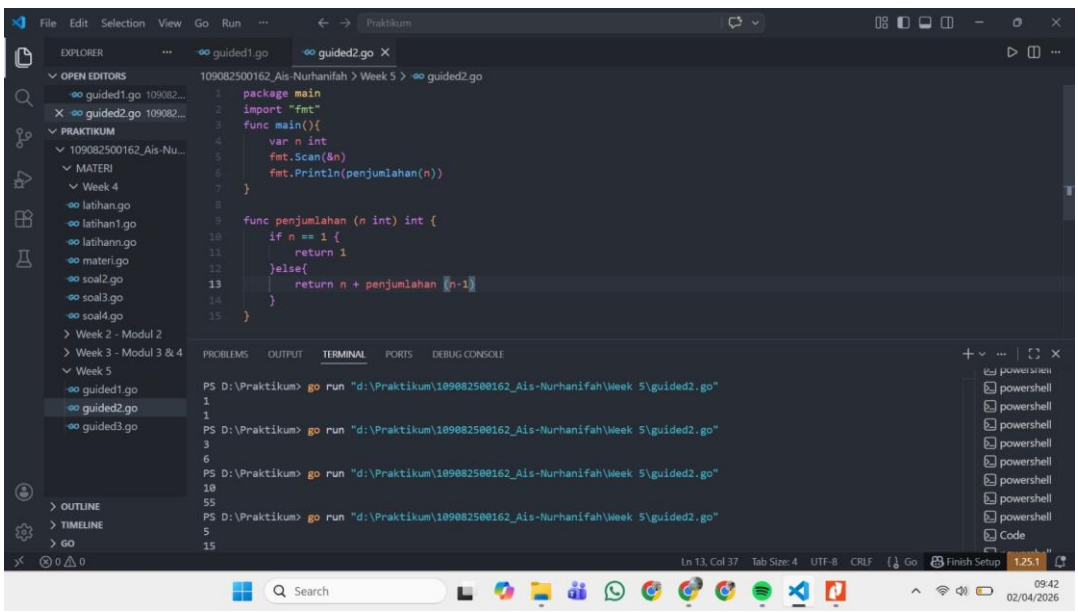
Program ini menggunakan teknik **rekursi**, di mana fungsi baris memanggil dirinya sendiri untuk memproses urutan angka secara menurun. Fokus utama dari program ini adalah menghasilkan urutan bilangan dari n hingga mencapai angka 1. Di dalam fungsi tersebut, terdapat dua komponen logika utama: **kondisi berhenti** (*base case*) dan **pemanggilan berulang** (*recursive step*). Kondisi berhenti akan tercapai saat variabel bilangan bernilai 1, yang menandakan bahwa proses pencetakan harus berakhir setelah angka terakhir tersebut ditampilkan.

Secara alur kerja, jika dilihat dari output pada gambar, misalnya saat saya memasukkan angka **3**, program mencetak angka 3 sebagai input awal, lalu fungsi baris(3) bekerja dengan mencetak angka 3 terlebih dahulu ke layar. Karena 3 belum mencapai 1, fungsi masuk ke bagian else dan memanggil dirinya kembali dengan nilai bilangan - 1, yaitu 2. Proses ini berulang: angka 2 dicetak, lalu fungsi memanggil dirinya lagi dengan nilai 1. Saat sampai pada angka 1, kondisi if bilangan == 1 terpenuhi, angka 1 dicetak, dan rangkaian rekursi berhenti total. Hal ini menyebabkan angka muncul berderet ke bawah mulai dari nilai input hingga angka 1, persis seperti yang terlihat pada terminal di gambar tersebut.

2. [hasilPenjumlahan.go]

```
package main
import "fmt"
func main(){
      var n int
      fmt.Scan(&n)
      fmt.Println(penjumlahan(n))
}

func penjumlahan (n int) int {
      if n == 1 {
            return 1
      }else{
            return n + penjumlahan (n-1)
      }
}
```

Output :



Penjelasan :

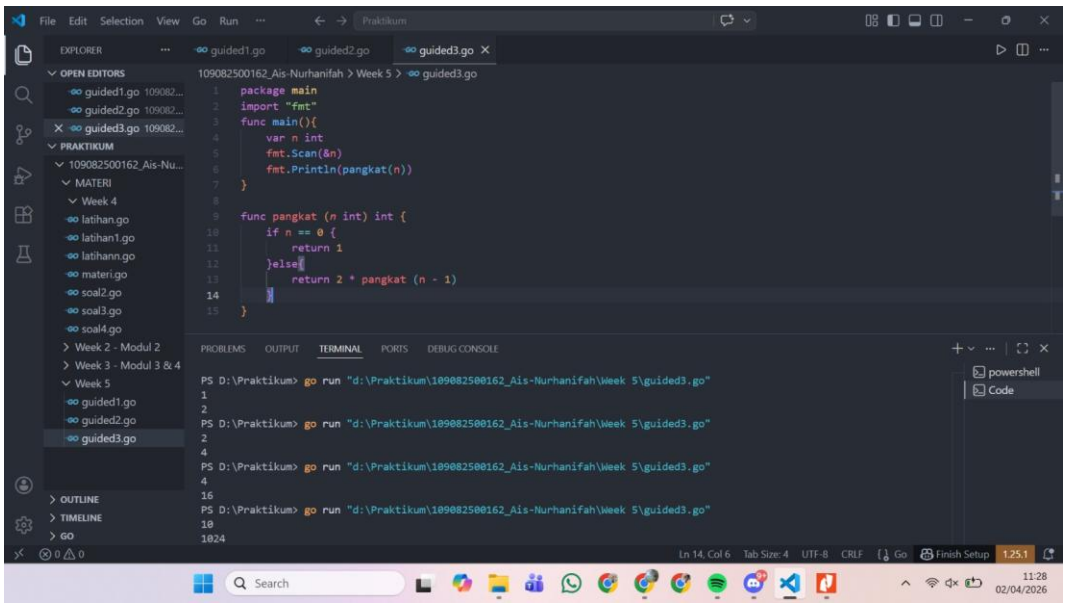
Program ini merupakan implementasi **rekursi** untuk menghitung hasil penjumlahan beruntun (deret aritmetika) dari angka n ke bawah hingga 1. Secara logika, fungsi penjumlahan bekerja dengan cara memecah masalah besar menjadi sub-masalah yang lebih kecil. Kondisi berhenti (*base case*) didefinisikan pada saat nilai n mencapai 1, di mana fungsi akan mengembalikan nilai 1. Jika n lebih besar dari 1, fungsi akan mengambil nilai n tersebut dan menambahkannya dengan hasil dari pemanggilan fungsi itu sendiri menggunakan nilai n-1.

Proses ini menciptakan rantai perhitungan yang akan "menumpuk" di memori hingga menyentuh angka 1, baru kemudian dijumlahkan seluruhnya saat kembali ke atas. Sebagai contoh, jika dilihat dari output pada gambar ketika diinput angka **3**, alur yang terjadi adalah 3 + (2 + 1), sehingga menghasilkan **6**. Begitu pula saat diinput angka **5**, program akan menghitung 5 + 4 + 3 + 2 + 1 yang menghasilkan **15**. Setiap angka yang muncul di terminal merupakan hasil akhir dari akumulasi nilai yang diproses secara rekursif tersebut. Perlu dicatat bahwa struktur if-else di sini sangat krusial agar program tidak berjalan selamanya (infinite recursion) ke angka negatif.

3. [mencariPangkat.go]

```
package main
import "fmt"
func main(){
      var n int
      fmt.Scan(&n)
      fmt.Println(pangkat(n))
}

func pangkat (n int) int {
      if n == 0 {
            return 1
      }else{
            return 2 * pangkat (n - 1)
      }
}
```

Output :



Penjelasan :

Program ini merupakan implementasi fungsi **rekursi** untuk menghitung perpangkatan dengan basis 2 (2^n). Secara logika, fungsi pangkat bekerja dengan terus mengalikan angka 2 sebanyak n kali. Kondisi berhenti (*base case*) ditentukan saat nilai n mencapai 0, di mana fungsi akan mengembalikan nilai 1 (karena secara matematis 2^0 = 1). Selama n belum bernilai 0, fungsi akan terus memanggil dirinya sendiri dengan argumen n-1, menciptakan rangkaian perkalian yang tertunda hingga menyentuh kondisi dasar.

Alur kerja ini dapat dilihat dari hasil output pada terminal. Sebagai contoh, ketika pengguna memasukkan angka **4**, fungsi akan melakukan proses 2 x (2 x (2 x (2 x 1))), yang menghasilkan output **16**. Begitu juga saat dimasukkan angka **10**, program melakukan rekursi sebanyak sepuluh tingkat hingga menghasilkan nilai **1024**. Struktur if-else pada kode ini memastikan bahwa setiap pemanggilan fungsi membawa program selangkah lebih dekat ke angka nol, sehingga mencegah terjadinya pengulangan tanpa henti (*infinite recursion*) yang dapat menyebabkan program eror atau *crash*.

C. Unguided

1. Soal Unguided 1

   Deret fibonacci adalah sebuah deret dengan nilai suku ke-0 dan ke-1 adalah 0 dan 1, dan nilai suku ke-n selanjutnya adalah hasil penjumlahan dua suku sebelumnya. Secara umum dapat diformulasikan Sn = Sn-1 + Sn-2. Berikut ini adalah contoh nilai deret fibonacci hingga suku ke-10. Buatlah program yang mengimplementasikan fungsi rekursif pada deret fibonacci tersebut.

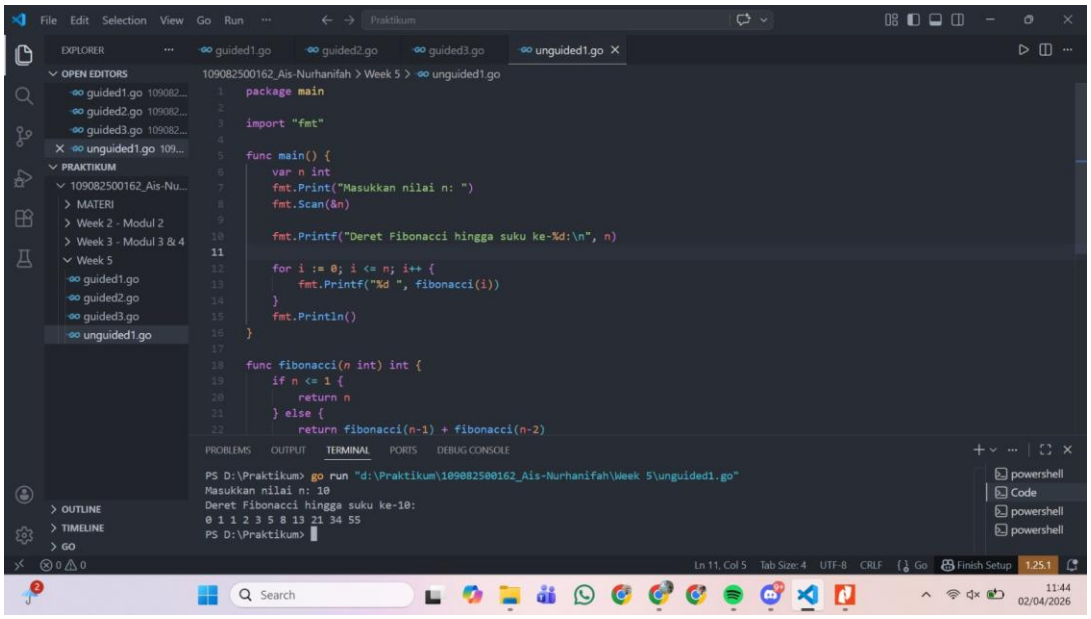
n	0	1	2	3	4	5	6	7	8	9	10
S_n	0	1	1	2	3	5	8	13	21	34	55

Jawaban :

```
package main

import "fmt"

func main() {
    var n int
    fmt.Print("Masukkan nilai n: ")
    fmt.Scan(&n)
```

```
        fmt.Printf("Deret Fibonacci hingga suku ke-%d:\n",
    n)

        for i := 0; i <= n; i++ {
            fmt.Printf("%d ", fibonacci(i))
        }
        fmt.Println()
    }

    func fibonacci(n int) int {
        if n <= 1 {
            return n
        } else {
            return fibonacci(n-1) + fibonacci(n-2)
        }
    }
```

Output :



Penjelasan :

program tersebut merupakan implementasi untuk menampilkan deret **Fibonacci** menggunakan teknik **rekursi**. Fungsi fibonacci dirancang untuk menghitung nilai suku berdasarkan aturan matematika di mana setiap angka adalah hasil penjumlahan dari dua angka sebelumnya (Sn = S{n-1} + S{n-2}). Kondisi berhenti (*base case*) ditentukan saat nilai n kurang dari atau sama dengan 1, di mana fungsi akan langsung mengembalikan nilai n itu sendiri (0 atau 1), guna mencegah pemanggilan fungsi tanpa akhir.

Di dalam fungsi main, program menerima input dari pengguna untuk menentukan batas suku yang akan ditampilkan. Program kemudian menjalankan perulangan for mulai dari indeks 0 hingga n, di mana pada setiap perulangannya, fungsi fibonacci dipanggil untuk menghitung nilai suku tersebut. Hasilnya, seperti yang terlihat pada terminal di gambar, ketika pengguna memasukkan angka **10**, program akan mencetak barisan angka secara horizontal mulai dari 0 hingga 55. Pendekatan ini secara visual membuktikan bagaimana setiap angka dalam deret terbentuk dari akumulasi pemanggilan rekursif yang memecah masalah hingga ke nilai dasarnya.

2. Soal Unguided 2

   Buatlah program yang mengimplementasikan rekursif untuk menampilkan faktor bilangan dari suatu N, atau bilangan yang apa saja yang habis membagi N.

   Masukan terdiri dari sebuah bilangan bulat positif N

   Keluaran terdiri dari barisan bilangan yang menjadi faktor dari N (terurut dari 1 hingga N ya)

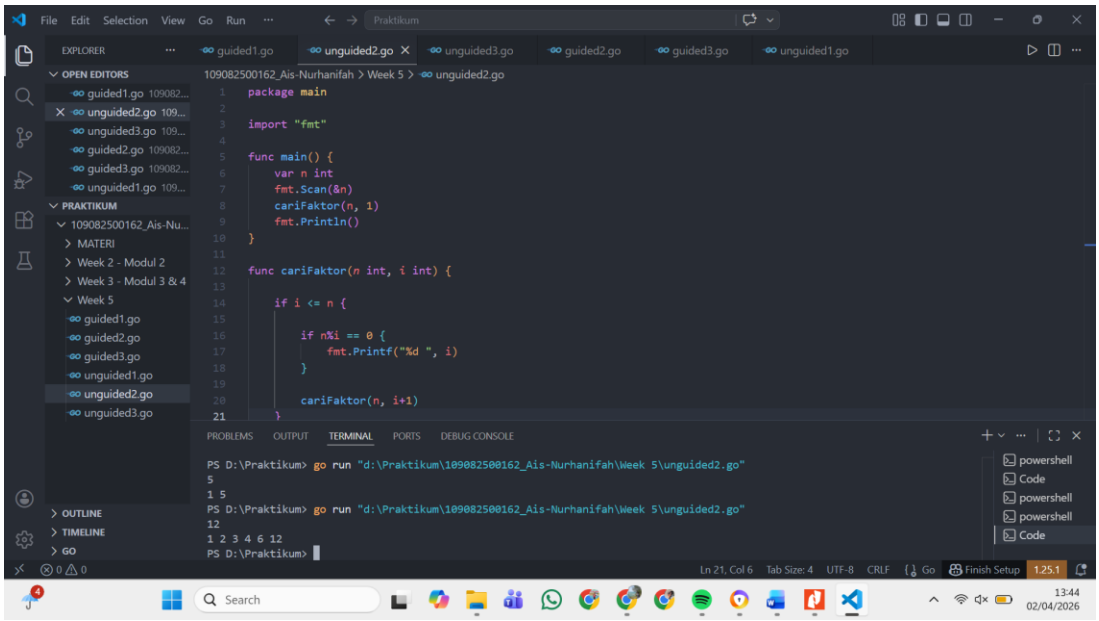
   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5	1 5
2	12	1 2 3 4 6 12

   Jawaban :

```
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    cariFaktor(n, 1)
    fmt.Println()
}

func cariFaktor(n int, i int) {

    if i <= n {

        if n%i == 0 {
            fmt.Printf("%d ", i)
        }

        cariFaktor(n, i+1)
    }
}
```

Output :



Penjelasan :

program tersebut merupakan implementasi **rekursi** untuk mencari dan menampilkan semua faktor dari sebuah bilangan bulat positif N. Faktor bilangan didefinisikan sebagai angka-angka yang dapat membagi habis N tanpa menyisakan sisa pembagian. Fungsi cariFaktor bekerja dengan menggunakan dua parameter utama: n sebagai bilangan target yang akan dicari faktornya, dan i sebagai angka pembagi yang diuji secara bertahap mulai dari angka 1. Strategi yang digunakan adalah membandingkan hasil operasi modulo (n % i) dengan angka 0; jika hasilnya benar nol, maka angka i tersebut adalah faktor dan akan langsung dicetak ke layar.

Proses rekursif ini dimulai dengan pemanggilan awal cariFaktor(n, 1) dan akan terus berlanjut dengan memanggil dirinya sendiri sambil menambah nilai i sebesar 1 pada setiap tahapnya (i + 1). Kondisi berhenti (*base case*) dari fungsi ini terjadi ketika nilai pembagi i sudah melampaui nilai n, sehingga fungsi akan berhenti memanggil dirinya sendiri secara otomatis. Dengan pendekatan ini, program secara sistematis menguji setiap angka dari 1 hingga N secara berurutan. Penggunaan fmt.Printf memastikan bahwa daftar faktor yang ditemukan tercetak menyamping dalam satu baris, sehingga menghasilkan urutan faktor dari yang terkecil hingga yang terbesar.

3. Soal Unguided 3

Buatlah program yang mengimplementasikan rekursif untuk menampilkan barisan bilangan ganjil.

Masukan terdiri dari sebuah bilangan bulat positif N.
Keluaran terdiri dari barisan bilangan ganjil dari 1 hingga N.

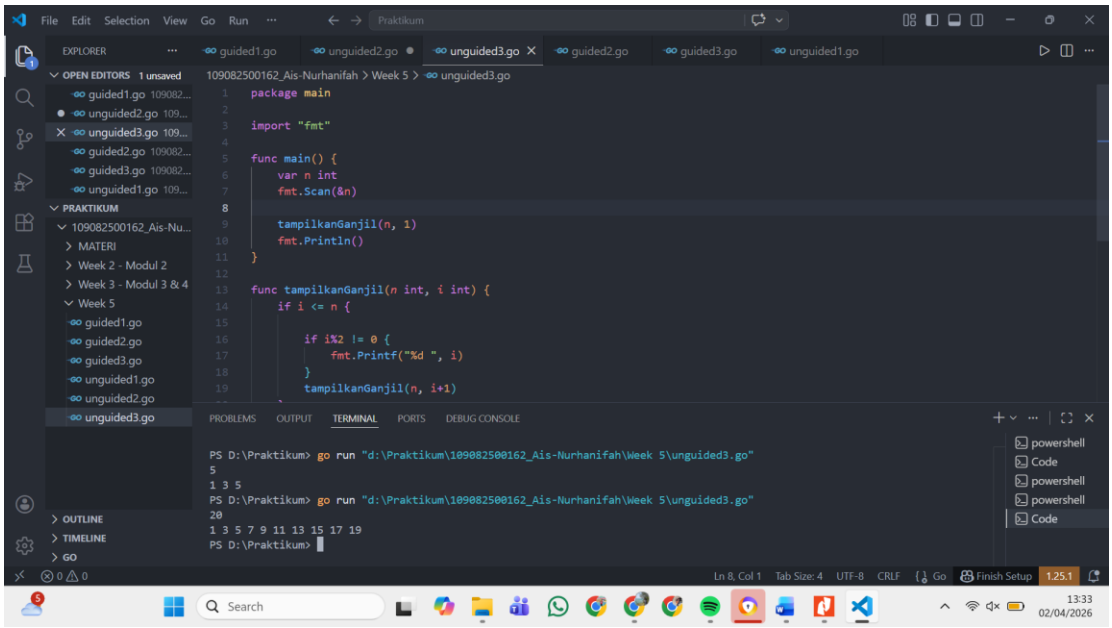
Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5	1 3 5
2	20	1 3 5 7 9 11 13 15 17 19

Jawaban :

```
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)

    tampilkanGanjil(n, 1)
    fmt.Println()
}

func tampilkanGanjil(n int, i int) {
    if i <= n {

        if i%2 != 0 {
            fmt.Printf("%d ", i)
        }
        tampilkanGanjil(n, i+1)
    }
}
```

Output :



Penjelasan :

program tersebut merupakan implementasi **rekursi** untuk menampilkan barisan bilangan ganjil dari angka 1 hingga n. Fungsi tampilkanGanjil bekerja dengan menggunakan dua parameter utama: n sebagai batas akhir angka yang dimasukkan pengguna, dan i sebagai angka pencacah yang sedang diperiksa. Strategi yang digunakan adalah melakukan pengecekan sisa bagi menggunakan operator modulo (i % 2 != 0); jika angka tersebut tidak habis dibagi dua, maka angka tersebut dikategorikan sebagai bilangan ganjil dan langsung dicetak ke layar.

Alur kerja rekursi ini dimulai dari angka 1 dan akan terus memanggil dirinya sendiri dengan menambahkan nilai i sebesar 1 pada setiap tahapnya (i + 1). Kondisi berhenti (*base case*) didefinisikan dengan instruksi if i <= n, yang berarti program akan berhenti melakukan pemanggilan fungsi secara otomatis ketika angka pencacah sudah melewati batas yang ditentukan. Seperti yang terlihat pada terminal di gambar, ketika pengguna memasukkan angka **5**, program menghasilkan output **1 3 5**, dan saat memasukkan angka **20**, program menampilkan deret ganjil hingga **19**. Teknik ini memastikan bahwa barisan bilangan ganjil ditampilkan secara berurutan dan efisien tanpa menggunakan perulangan iteratif tradisional seperti for atau while.

D. Kesimpulan

Dapat disimpulkan bahwa teknik rekursi merupakan metode pemrograman yang sangat efektif untuk menyelesaikan persoalan dengan pola berulang atau sub-struktur yang serupa. Melalui implementasi pada berbagai kasus seperti baris bilangan menurun, penjumlahan deret, hingga perhitungan perpangkatan, terlihat bahwa fungsi rekursif mampu memecah masalah besar menjadi tahapan yang lebih sederhana. Penggunaan bahasa Go dalam praktikum ini menunjukkan bahwa struktur fungsi yang memanggil dirinya sendiri memberikan alur logika yang bersih dan deklaratif dibandingkan menggunakan perulangan iteratif tradisional.

Penerapan kondisi berhenti (*base case*) menggunakan struktur *if-else* terbukti menjadi elemen paling krusial dalam setiap program yang dibuat. Tanpa adanya *base case* yang tepat, program akan terjebak dalam pengulangan tanpa henti (*infinite recursion*) yang dapat menyebabkan kegagalan sistem atau *crash*. Hal ini terlihat jelas pada soal-soal *Unguided*, di mana penentuan nilai dasar pada deret Fibonacci atau batas akhir pada pencarian faktor bilangan menjadi penentu utama akurasi *output* yang dihasilkan di terminal.

Secara keseluruhan, praktikum ini berhasil membuktikan bahwa rekursi dapat digunakan baik untuk operasi akumulatif maupun pencarian sistematis secara berurutan. Program mampu menghasilkan deret Fibonacci yang akurat, daftar faktor bilangan yang terurut, hingga barisan bilangan ganjil dengan efisien. Dengan memahami keseimbangan antara *recursive step* dan penggunaan memori *stack*, mahasiswa dapat mengoptimalkan kode program untuk menyelesaikan logika matematika yang kompleks dengan penulisan yang lebih ringkas dan mudah dipahami.